

40. Objektová orientace (základní koncepty, třídě a prototypově orientované jazyky, OO přístup k tvorbě SW)

40. Objektová orientace

SZZ okruh 40 (FIT BUT). Modelování reality objekty a jejich komunikací.

Shrnutí

Základní koncepty

- **Abstrakce** (objekt = atributy + metody, skrytí detailů za rozhraní), **zapouzdření** (viditelnost public/protected/private), **dědičnost** (generalizace/specializace, znovupoužití), **polymorfismus** (stejná zpráva různým objektům).
- Klasifikace jazyků: třídě × beztřídě (prototypově); čisté × hybridní; staticky × dynamicky typované; jednoduchá × vícenásobná dědičnost.

Třídě jazyky

- **Třída** = předloha/datový typ; **objekt** = instance (identita + reference na třídu + data atributů).
- Vazby: statická (žádná), **brzká** (compile-time), **pozdě** (run-time → polymorfismus přes **VMT** — tabulka virtuálních metod, jedna na třídu).
- Vícenásobná dědičnost (problém kolizí — často zakázána); **rozhraní** = náhrada vícenásobné dědičnosti.

Prototypově jazyky

- Tvorba objektů **klonováním prototypu**; dědičnost přes **prototype chain**; volání zděděných metod přes **delegaci**; polymorfismus předefinováním metody (JS, Lua).

OO přístup k tvorbě SW

- Návrh přes [[okruhy/34-uml|UML]] (diagramy tříd), znovupoužití dědičností, override (polymorfismus), **ORM** pro ukládání do relační DB, **návrhové vzory** (Factory, Observer, Singleton, MVC).

Modelování v UML viz [[okruhy/34-uml]]; ADT a struktury viz [[okruhy/29-datove-ridici-struktury]]; OO ve vývoji viz [[okruhy/33-zivotni-cyklus-sw]].

Časté otázky u zkoušky

Na co se u tohoto okruhu typicky ptají. Plné odpovědi níže.

Základní koncepty → [[#Základní koncepty]] - Čtyři principy OOP? → abstrakce, zapouzdření, dědičnost, polymorfismus. - Třída vs. objekt? → Třída = předloha (**datový typ** — pozor, neříkat „šablona”);

objekt = konkrétní instance. - *Zapouzdření, viditelnost?* □ public (kdokoli), protected (třída + potomci), private (jen třída).

Dědičnost a polymorfismus □ [[#Třídní jazyky]] - *Polymorfismus, jak je realizován?* □ Stejná zpráva různým objektům; u překládaných přes **VMT** (pozdní vazba). - *Přetěžování vs. override?* □ Overloading = stejné jméno, jiné parametry; override = redefinice zděděné metody.

Třídní vs. prototypové □ [[#Prototypové jazyky]] - *Rozdíl?* □ Třídní tvoří instance z tříd; prototypové klonují objekty (delegace, prototype chain).

Rozhraní □ [[#Třídní jazyky]] - *Rozhraní vs. dědičnost?* □ Rozhraní deklaruje metody (bez implementace), náhrada vícenásobné dědičnosti; třída může implementovat více rozhraní □ polymorfismus mimo dědičnost.

Plné znění (ke studiu)

Objektově orientované programování (OOP) je programovací paradigma, které se začalo hojně objevovat v 80. letech minulého století. Základem tohoto paradigmatu je abstrahovat a modelovat principy reálného světa pomocí objektů a jejich vzájemné komunikace. V dnešní době je toto paradigma jedno z nejrozšířenějších, zejména pro velké aplikace. Mezi **výhody** patří **analogie reálného a softwarového modelu, flexibilita, znovupoužitelnost**. Nevýhodou může být složitější sémantika, delší učící se křivka či rezie spojená s prací s objekty. Mezi základní koncepty OOP patří:

- **Abstrakce** (Abstraction): pomocí objektů, skrytí implementačních detailů za rozhraní,
- **Zapouzdření** (Encapsulation): pomocí viditelnosti atributů,
- **Mnohotvárnost** (Polymorphism): abstraktní funkce a VMT (Virtual Method Table),
- **Dědičnost** (Inheritance): generalizace/specializace.

Klasifikace objektově orientovaných jazyků

- Dle přístupu k vytváření objektů:
 - **Beztrídní** OOJ
 - * JavaScript (od es6 má sice keyword “class” pro definici tříd, ale v pozadí se pořád používají prototypy)
 - * Lua - prototype-based, class-less,
 - **Třídní** OOJ
 - * C#
 - * C++ - class-based
- Dle čistoty objektového modelu:
 - **Čisté** (Ruby, Python - vše je objekt),
 - **Hybridní** (C#, C++ - míchání s jinými paradigmaty, doplněny objekty),
- Dle platformy pro běh OO programů:
 - **Překládané** (C++ - efektivita běhu, více zdrojového textu),
 - **Interpretované** (Python, PHP - pomalé, velmi flexibilní),
 - **Částečně interpretované** (C#, Java - bytecode, mezikód, virtuální stroj)
- Dle dědičnosti (počet přímých předků):
 - **Jednoduchá** (C#, Java - oba tyto jazyky ale podporují vícenásobnou dědičnost rozhraní),

- **Vícenásobná** (C++, Python - problematická, nutno řešit kolize)
- Dle předmětu dědění:
 - **Dědičnost implementace** (C++, C#, Java, Python, ...)
 - * Třídní dědičnost: nadtřída (superclass), podtřída (subclass)
 - * Delegace
 - **Dědičnost rozhraní** (C#, Java)

Klasifikace nejen OOJ

- Dle způsobu určení typů:
 - **Beztypové** (Lambda kalkul - nemají žádný typ),
 - **Netypované** (Python, JavaScript - uživatel nepracuje s typem, typ ale mají a lze na vyžádání zjistit),
 - **Typované** (C++, C# - uživatel explicitně specifikuje typ, nebo musí být odvoditelný)
- Dle důslednosti kontroly typů:
 - **Silně typované** (Rust, Go - nelze provádět implicitní konverze, type-safe),
 - **Slabě typované** (C, C++ (méně než C) - implicitní konverze)
- Dle doby kontroly typů:
 - **Staticky typované** (C, C++ - při překladu, tedy už před samotným spuštěním programu),
 - **Dynamicky typované** (Python, JavaScript - za běhu, run-time).

Minimální model OO výpočtu

Minimální výpočetně úplný OO model výpočtu potřebuje pouze:

- proměnné,
- konstrukt objektu (způsob vytváření objektů),
- zasílání zpráv (komunikace mezi objekty).

Základní koncepty OOP

Tyto koncepty by měly být více méně shodné pro každý OOJ.

Abstrakce - Objekt

Objekt je **autonomní výpočetně úplná** entita, obvykle je tvořena **atributy** a **metodami**. Identita objektu ale nezávisí na jeho atributech. U třídních jazyků jsou objekty **instancemi tříd**.

Kompozice Objekt může obsahovat jako položky i jiné objekty.

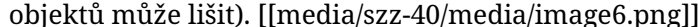
Zapouzdření

Jedná se o **uzavřenost** vůči okolním objektům, dnes je to obvykle implementováno pomocí **identifikátorů viditelnosti** (**public**, **protected**, **private**). V případě **zpráv** mluvíme o přístupu přes:

- **veřejný** protokol umožňuje okolním objektům zasílat zprávy tomuto objektu. Zaslání zprávy veřejným protokolem může vést na:
 - **chybu** - objekt nerozumí zprávě,
 - **invokaci** odpovídající metody a **navrácení** výsledné hodnoty odesílateli.
- **interní** protokol se používá, když objekt zasílá **zprávu sám sobě** (this, self). Zaslání zprávy interním protokolem může vést na:
 - **chybu** - objekt nerozumí zprávě,
 - **přístup k atributu** (čtení nebo zapsání),
 - **invokaci** odpovídající metody a návratu výsledku.

Zpráva Zpráva je tvořena z **příjemce** (objekt, kterému je zpráva zaslána), **selektoru** (metody) a **argumentů**.

Polymorfismus (Mnohotvárnost)

Vychází z toho, že **stejnou zprávu lze zaslat různým objektům** (často je to ale **omezeno** typovým systémem, aby nedocházelo k chybám typu “nerozumím zprávě” a stejné zprávy lze zasílat různým objektům jen v rámci dědičnosti). Protokol umožňuje **individuální reakci** na zaslání zprávy (volající díky zapouzdření nezná implementaci zprávou invokované metody, implementace se u různých objektů může lišit). 

Dědičnost

Vychází z toho, abychom **nemuseli** neustále **opakovat implementaci** podobných vlastností. Jedná se o jednu z hlavních předností OOP, a to **znovupoužitelnost**. Dědičnost umožňuje **sdílení** společných **položek** (atributy a metody) **od předků** a možnost definovat **individuální položky v potomcích**. Zděděný (specializovaný) objekt (třída) tak sdílí všechny atributy a metody předka (jejich použití objektem závisí na viditelnosti - musí být **public** nebo **protected**) a dále může být specializován:

- **přidáním** nových atributů a metod,
- **modifikací** metod (**override**),
- někdy lze také zakázat některé položky, většinou ale **ne**, viz dále.

Zahrnutí typu (subsumption) Pokud B je podtřída třídy A, lze instanci třídy B využít kdekoli je očekávána instance třídy A. Pak se provádí přístup k **objektu třídy B přes protokol A**.

Problém vícenásobné (třídní) dědičnosti

- nadtřídy obsahují položky **stejného jména a typu**,
- nadtřídy obsahují položky **stejného jména**, ale **různých typů**,
- **inicializace instancí** - pořadí volání konstruktorů,
- **uložení instancí v paměti** - instance třídy C (C je přímá podtřída A i B) lze využít **kdekoli**, kde očekávám instance tříd A nebo B.

Řešení problémů vícenásobné (třídní) dědičnosti Nejjednodušší je vícenásobnou dědičnost **zakázat**, stejně se ukázalo, že není často při programování nutná. Pokud ji ale potřebujeme:

- **Metoda stejného jména a typu:** zakázat, použít první výskyt (dle pořadí zapsání dědění v kódu), programátor musí explicitně vybrat jednu, skrytí (např. A::m0) přístupná jen v A, ne v C).
- **Metody stejného jména a různých typů (parametrů):** zakázat, povolit přetěžování metod (overloading), lze ale pouze u metod s rozdílnou signaturou (návratová hodnota není součástí signatury), takže s rozdílnými parametry.
- **Atributy stejného jména a typu:** zakázat, skrytí a existence obou (např. A::d přístupný jen v A, ne v C), sloučení.
- **Atributy stejného jména a různého typu:** zakázat, nebo ponechat, ale při práci s objektem musí být možné atributy vždy odlišit.

Tvorba nových objektů

- Vytvoření **prázdného objektu** a přidání položek (atributů a metod).
- Klonování (kopie) a přidání či úprava položek. Klonovanému objektu říkáme **prototyp**. Kopie může být **mělká** (pouze 1. úroveň atributů), nebo **hluboká** (celý objekt). Princip klonování používají beztřídní OOJ.
- Vytvořením pomocí **předlohy - třídy** a naplnění atributů hodnotami. Děje se tak pomocí **konstruktoru**. Pro každý objekt je definována jeho třída - předloha. Třída také může být objektem první kategorie.

Vzájemné vazby mezi třídami/objekty

Řeší se přes **ukazatele/reference** a **dopřednou deklaraci**. Dopředná deklarace se poté používá i pro metody a ty se definují mimo tělo funkce, aby v metodách bylo možné odkazovat atributy cyklicky závislých proměnných.

Třídně orientované jazyky

Třídně orientované jazyky využívají pro tvorbu nových objektů - **instancí** šablony, které nazýváme **třídou**. Třída může být sama o sobě objekt nebo pouze entita, která obsahuje:

- seznam **instančních atributů** (atributů, které bude mít objekt po vytvoření) včetně **metadat**,
- **data třídních atributů** (pokud je třída taky objekt),
- implementace **instančních metod** (sdílené mezi instancemi),
- reference na její třídu,
- **statické** (!= třídní) položky - **atributy** a **metody** (pokud je třída entitou jazyka a není objekt).

Instance - objekt je poté tvořen svojí **identitou**, **referencí na třídu**, **daty instančních atributů**.

Třída jako objekt první kategorie

Třída je také objekt a pracuje se s ní analogicky jako s objekty, tj. **zasíláním zpráv**. Ve třídních metodách lze použít **self/this** parameter pro přístup k objektu třídy.

Třídní dědičnost

Stejný význam jako dědičnost na úrovni objektů (ve většině jazyků používáme hlavně třídní dědičnost). Dědičnost tříd je šablonou pro dědičnost vzniklých (instanciovaných) objektů. Hlavní účel je **sdílení/znovupoužití** položek skrz třídy, **rozšíření** o nové položky a **redefinice** u specializovaných tříd.

Hierarchie třídní dědičnosti

Matematicky můžeme mluvit o dědičnosti jako **relaci částečného uspořádání na množině tříd**. Hierarchie třídní dědičnosti je poté tvořena grafem relace dědičnosti.

Levý obrázek ukazuje dědičnost u jazyků pouze s **jednoduchou** dědičností, na pravém je znázorněna vícenásobná dědičnost (více přímých předků pro třídu Y). 



Statické volání funkce (žádná vazba)

Jedná se o běžnou metodu/funkci, která je pouze ve jmenném prostoru třídy (její jediný rozdíl od normální funkce). Uvnitř metody **nelze** používat **self/this** parameter, není implicitně předán. Může ale používat jiné statické proměnné třídy.

Brzká (časná) vazba

Při překladu je jasné, jaká implementace bude volána, **není** třeba mít **odkaz na metodu** v konkrétním objektu, překladač určí volání metody na základě jeho typu. V metodách lze odkazovat objekt, nad kterým je metoda volána pomocí **self/this** parametru, který je do metody implicitně předán. (ve skutečnosti překladač převede zápis `obj.call()` na `call(obj)`)

Pozdní vazba

Umožňuje **polymorfismus** (volání virtuálních metod), volání metody nad objektem se řeší **až za běhu** dle jeho konkrétního typu. Stejně jako u brzké vazby lze v polymorfních metodách odkazovat objekt nad kterým je metoda volána (**self/this**). U překládaných OoJ se řeší pomocí **tabulky virtuálních metod** (VMT).

Implementační problémy

Problémy implementace vychází z toho, že OoJ používají **objektovou paměť**, která je **asociativní** a **heterogenní** a modelem výpočtu je **zasílání zpráv** objektům (invokace metod). Paměť počítače je ale **homogenní** a **neasociativní** a modelem výpočtu je **volání instrukcí** a použití **zásobníku**. Je nutné řešit:

- **Uložení instancí a přístup k atributům**, tak aby byla reflektovaná dědičnost. Problematické je zejména zajistit přístup k atributům objektu při vícenásobné dědičnosti, tak aby šlo k objektu, který dědí z více objektů, přistupovat protokolem každého z obecných objektů. U jednonásobné dědičnosti to nemusí být problém, hodnoty specializovaných atributů se ukládají za hodnoty obecných atributů a při přístupu se specializované atributy ignorují (paměť se přetypuje na obecný objekt). Ale jak uložit atributy při mnohonásobné dědičnosti? Pouhé “přetypování” nebude fungovat.
- **Uložení a invokace polymorfních metod**. Ukládání kódu metody v objektu je **nesmysl**, stačí metodu ukládat **jednou**, např. ve třídě a objekt implicitně předávat jako parametr (nultý parametr) při volání metody. Do objektu by tedy mohlo stačit uložit **odkazy na metody**, což **řeší i**

polymorfismus tj. každý objekt má odkazy na takové metody, které doopravdy implementuje. Tento způsob je jednoduchý ale má 2 zásadní problémy:

- **přístup k metodám předků** (pomocí **super/base**) po jejich redefinici - objekt by musel obsahovat odkazy na jeho redefinici metody, všechny různé redefinice metod předky a originální definici metody,
- **plýtvání pamětí** - při vytvoření tisíce stejných objektů bude v paměti uloženy zbytečně tisíce stejných odkazů na metody.

Řešením problémů je použití **tabulky virtuálních metod** (virtual method table - VMT), kterou odkazuje objekt na místo všech jeho metod.

Tabulka virtuálních metod (VMT; Virtual Method Table)

Je to opravdu **tabulka virtuálních metod** (NE virtuální tabulka). Pro každou třídu (typ objektu) existuje **pouze jednou** a všechny instance (objekty) této třídy na ní odkazují. Dědičnost je řešena tak, že z VMT specializovaného objektu vede odkaz na VMT jeho předka a z VMT předka odkaz na VMT dalšího předka atd.

Tento princip **zanořeného** odkazování je ale **pomalý**, proto se za cenu mírného zvýšení spotřeby paměti tabulka pro danou třídu odkazuje **všechny polymorfní metody**, i když nebyly přepsány (**flat table**). Tabulka **neobsahuje** odkaz na **rodičovskou** tabulku. Rodičovskou tabulku při volání přes **self/this** dokáže **určit překladač** na základě typu self/this, protože každé **tabulce přiděluje statickou adresu**. 



Vytváření instancí (objektů)

Většinou se provádí pomocí klíčového slova **new** a je tvořena ze 2 kroků:

1. **Přidělení paměti** (většinou na hromadě) pro danou instanci (objekt).
2. **Inicializace atributů** pomocí **konstruktoru** (speciální metoda), nutné si v konkrétním jazyce zjistit, jaké je pořadí volání konstruktorů při dědičnosti.

Rušení instancí (objektů)

Při rušení objektů se volá speciální metoda tzv. **destruktor** (opět je nutné znát pořadí volání destruktorů při dědičnosti) a může být provedena:

- **Implicitně** pomocí správce paměti (**Garbage Collector**) objekty jsou z paměti mazány, když již jsou nedosažitelné. Programátor **nemá pod kontrolou**, kdy se tak stane a jak dlouho to zabere ☐ nelze použít např. u real time systémů s požadovanou odezvou. Algoritmy uvolňování ale obvykle **nezpůsobují** memory leaks.
- **Explicitní** použitím klíčového slova **delete** (nebo jiného) řeší **programátor** a má nad uvolňováním paměti **plnou kontrolu**, nicméně **může** vést na **memory leaks**.

Reflektivita/Reflexe (Reflection)

Vlastnost jazyků (interpretovaných a částečně interpretovaných), která umožňuje:

- zkoumat vnitřní reprezentaci (**Introspection**) entit programu (objektů),
- měnit vnitřní reprezentaci (méně časté).

Na základě prováděných změn a zkoumání vnitřní struktury dělíme reflektivitu na:

- **Strukturální:** pracuje se **statickými strukturami** (balíčky, třídy, metody), např. zjišťujeme názvy atributů objektu.
- **Behaviorální:** pracuje s **prováděním programu** (invokace metody, přiřazení, zásobník volání), např. voláme metody na základě hodnoty řetězce, který obsahuje její název.

Reflektivita umožňuje jednodušeji vytvářet generický software. Nejlepším příkladem je serializace a deserializace. Na základě reflexe můžeme zjistit názvy atributů, získat jejich hodnoty a vytvořit např. JSON reprezentaci. Taková funkce pak může mít za parametr libovolný objekt a nepotřebujeme mít speciální funkci pro každý objekt.

Typy vs. Třídy

- **Typ** udává množinu validních hodnot a operací nad nimi,
- **Třída** udává množina vnitřních stavů a operací nad nimi.

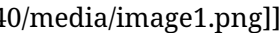
Typy jsou obecnější než třídy. Třída často určuje typ, ale ne naopak. Typ třídy může být určen:

- **jménem** např. int, Car, Person, ... Pak pokud máme funkci s parametrem **typu A**, můžeme ji volat s:
 - **instancemi** třídy **A**,
 - **instancemi** libovolné **podtřídy A** (kompatibilní podtyp).

Pro zajištění kompatibilního typu musíme použít dědičnost - **vyžádaná dědičnost** a **polymorfismus** se vyskytuje také **pouze v rámci dědičnosti**.

- **výčtem položek** tj. nezáleží, jestli jde o třídu Car nebo Person. Typ se zjistí prozkoumáním položek instance - **test implementace** potřebného podtypu:
 - **staticky** během překladu (často se k tomu využívá rozhraní/interface),
 - **dynamicky** za běhu.

Umožňuje **polymorfismus nezávislý na třídní dědičnosti**, např. funkce má jako parameter typ JmenoStari, která obsahuje dva atributy "jméno" a "stáří", pokud Car i Person mají (i mimo jiné) tyto atributy, lze je použít jako parametr pro volání této funkce (obdobně s metodami a kombinací metod a atributů).

- **kombinace** např. vícenásobnou dědičností nebo rozhraním 



Implementace skutečného **podtypu** je **náročná** (změna pořadí položek atd. kód metody je ale statický a očekává položky na stejných pozicích, ...) je nutné skutečný podtyp řešit **až na běhu**. Skutečný podtyp se tak typicky vyskytuje u **interpretovaných** (nebo alespoň částečně interpretovaných) a **dynamicky typovaných** jazyků. Objekt je ve skutečnosti **slovník** s názvem položky (klíč), hodnotou, typem, aj. a interpret hledá, jestli ve slovníku existují dané položky. U **překládaných a staticky typovaných** jazyků lze podtyp nahradit **rozhraním**.

Rozhraní

Jedná se o **náhradu vícenásobné třídní dědičnosti a skutečného podtypu**. Rozhraní je schéma, které deklaruje sadu metod. **Nelze** vytvářet **instance rozhraní**, ale rozhraní se **přiřazuje třídám**, které jej implementují (třída, které je přiřazeno rozhraní, musí implementovat všechny metody, které rozhraní deklaruje). Deklarace atributů v rozhraní je pouze teoretická a není většinou podporována (v C# lze v rozhraní specifikovat tzv. properties s { get; set; }, to ale **nejsou atributy** jedná se vlastně o funkci, která pomocí get a set získává a nastavuje daný atribut - field v C#). **Rozhraní** lze

kombinovat s jednoduchou **dědičností** a třída může **současně implementovat více rozhraní**. Samotná rozhraní mohou také **dědit** (částečné uspořádání na rozhraních) a **případně i vícenásobně**.

Koncept **rozhraní** umožňuje **polymorfismus** i mimo třídní dědičnost. Libovolná třída může implementovat funkce daného rozhraní libovolně. Pokud poté přistupujeme k objektu pomocí rozhraní, je nám skryto, o jaký objekt se jedná, takže volaná metoda může mít libovolnou funkcionalitu (v rámci zachování logiky programu na to musíme dávat pozor, i.e. funkci implementujeme tak, aby měla očekávanou funkcionalitu např. podle jména).

Implementace rozhraní

Objekt (instance třídy) odkazuje na více VMT, pro **každé rozhraní jiná VMT**, respektive z VMT objektu jsou odkazy na VMT realizující rozhraní (odkazující na metody, které deklaruje rozhraní).

Beztrídní objektově orientované jazyky

[[media/szz-40/media/image8.png]]

U beztrídních jazyků je tvorba objektů závislá na **klonování prototypů**. Prototyp je také objekt. Např. v JS je prototypem každého objektu **Object** a ten už nemá další prototyp. Pomocí prototypů se realizuje v JS **dědičnost**, každý objekt má jeden **přímý** prototyp, ten pak může mít další přímý prototyp atd. (tzv. **prototype chain**). Obecně ale může mít objekt více přímých předků (prototypů). **Specializace** objektů je prováděna **dynamicky** za běhu (případně by mohla být prováděna staticky před překladem) přidáním nové položky (**atributu** nebo **metody**). Klonováním prototypu se **nekopírují metody** klonovaného objektu. **Atributy** jsou obecně **zkopírovány** a je jim při klonování nastavena buď zadaná hodnota, nebo implicitní hodnota (hodnoty v prototypu). Metody zůstávají pouze u prototypu. Invokace těchto metod u naklonovaného objektu se řeší pomocí **delegace** (výpočetní systém se nejdříve snaží metodu spustit na daném objektu, pokud ji nemůže najít, deleguje ji na prototyp). **Polymorfismus** je zajištěn tak, že u naklonovaného objektu **definujeme stejně pojmenovanou metodu**. Např. JS je **dynamicky typovaný netypovaný** jazyk a implementuje **skutečný podtyp**, což umožňuje i polymorfismus i mimo dědičnost. Pokud objekt neimplementuje danou metodu (nerozumí zprávě) a ani žádný z jeho prototypů, dochází k chybě. Příklad klonování z JS: [[media/szz-40/media/image10.png]]

personPrototype je zde objekt (složené závorky je mimo jiné způsoby v JS syntax pro tvorbu objektu), jeho prototypem je tedy v tomto případě Object (vznikl jeho klonováním, i když to není na první pohled zřejmé). U tvorby objektu* carl je již ale zřejmé, že se jedná o klonování objektu personPrototype. Objekt carl tak má za přímý prototyp personPrototype a ten má přímý prototyp Object. Voláním metody greet nad objektem carl vede na delegaci a volá se metoda greet objektu personPrototype.

- *pokud použijeme tento způsob klonování v JS nejsou zkopírovány ani atributy, ty také odkazuje na atribut prototypu, dokud není jeho hodnota v naklonovaném objektu přepsána.

Delegace

Delegace je obdoba zaslání zprávy. Zpráva je v tomto případě zasílána rodiči objektu (**příjemce** je rodič objektu) a kromě názvu metody (**selektoru**) a **parametrů** obsahuje i **objekt**, kterému byla zpráva prvně adresována (**původního příjemce**).

Sloty (v jazyce SELF)

Slot obsahuje **název položky** a **odkaz** na:

- **datový objekt**,
- **objekt s metodou**,

- **rodičovský objekt.**

Rysy (traits v jazyce SELF)

Rys je objekt, který obsahuje pouze sdílené **metody** a **rodičovské sloty** (neobsahuje jiné atributy/sloty), které umožňují delegaci. Prototyp pak obsahuje datové **sloty pro atributy** a jejich **implicitní hodnoty** (použité při klonování) a **rysy**, na které jsou **delegovány** zprávy (delegace) při volání “instančních” metod. Mezi **rysem a třídou** lze nalézt v tomto směru **analogie**, třída i rys nese instanční metody, které jsou z konkrétních instancí (objektů) odkazovány.

OO přístup k tvorbě SW

Při objektově orientované tvorbě SW využíváme skutečnosti, že **objekty reálného světa lze modelovat** (do jisté úrovně detailu) **objekty v SW**. A snažíme se využít základních vlastností OOJ, a to **abstrakci, zapouzdření, dědičnost a polymorfismus**. Např. v reálném světě potřebujeme vytvořit fakturu, což znamená, že dle nějakých konvencí ji musíme sepsat (např. vyplnit informace o dodavateli a odběrateli, položkách faktury, celkové ceně atd.). Fakturu poté můžeme odeslat buď poštou, nebo ji naskenovat a odeslat emailem.

Pokud by k nám přišel někdo, že už nechce psát faktury ručně a chce abychom mu pro to napsali SW, můžeme postupovat následovně (posloupnost kroků nemusí odpovídat realitě):

1. Setkáme se s touto osobou (zákazník) a na základě jeho požadavků můžeme pomocí **UML** (standardizovaný grafický jazyk podporující objektovou orientaci) vytvořit objektově orientovaný návrh např. pomocí **diagramů tříd** a **diagramů objektů**.
2. Následně zjistíme, jestli již **neexistuje** nějaká objektová orientovaná implementace, která řeší tvorbu faktur na základě **vytvořených diagramů**. Pokud najdeme vhodné již implementované **třídy** (objekty u beztrždního jazyka), které obsahují část atributů a metod. Můžeme použít tyto - využijeme jednu z výhod OOP, a to **znovupoužitelnost**.
3. Následně z vyhledaných objektů **zdedíme** (použijeme **dědičnost**) a do zděděného objektu **doplníme** potřebné atributy a metody (provedeme specializaci).
4. Metody, které jsou již v původním objektu implementovány, ale nevyhovují našemu zadání přepíšeme (**override**) a využijeme vlastnosti OOJ **polymorfismu**.
5. Pro ukládání faktur (tj. objektů faktura) můžeme využít buď **objektové databáze**, ale mnohem častější je využití SQL databází a vrstvy mezi databází a OOJ, která převádí relační data na objekty - **ORM (Objektově-relační mapování)**

Shrnutí

Při OO přístupu k vývoji SW se snažíme využít:

- **analogie** objektů reálného světa (mají nějaké znaky a funkce) se SW objekty (mají atributy a metody), Snažíme se zachovat vztah dat a jejich manipulátorů, respektive akcí nad nimi.
- při návrhu využíváme nástroje podporující objektový přístup, jako je jazyk **UML - formální reprezentace OOP**,
- využíváme **dekompozice** složitých objektů na jednodušší,
- při implementaci se snažíme využít **výhod OOP**, zejména **znovupoužitelnost**,
- při implementaci využíváme základní **koncepty OOP**, a to **abstrakci, zapouzdření, dědičnost a polymorfismus**.

- snažíme se využít nástroje, které umožňují převádět jiný SW na objektové paradigma, např. **ORM**.
- snažíme se využít **návrhových vzorů** (poskytují radu/návod/vzor, jak vhodně řešit často vyskytující se problémy a umožňují lepší znovupoužitelnost, např. singleton, factory, adapter, MVC, MVVM, MVP, Facade).

[[media/szz-40/media/image3.png]]

Zdroje

- SZZ okruh 40 — studijní materiály FIT BUT (szz-40.docx). Obrázky: media/szz-40/.

Navigace: [[index | • Přehled okruhů]] · □ [[okruhy/39-planovani-synchronizace-procesu | 39. Plánování a synchronizace procesů]] · další: [[okruhy/41-assembler | 41. Programování v JSI (assembler)]]
□